Lucas Kenzo Izumi
Lucas Cordeiro Gatti
Caio Brun de Oliveira

Análise de pilhas, deques e listas para criar um sistema de cartas de baralho

Estrutura de dados - SCC0223
Universidade de São Paulo
Instituto de ciências matemáticas e de computação - ICMC
30 de outubro de 2026

1 PROJETO DE SOLUÇÃO

A implementação escolhida para o baralho foi uma lista duplamente encadeada e um deque duplamente encadeado para o histórico.

A operação VIRA percorre o baralho utilizando um for loop, partindo de uma referência fixa à carta inicialmente posicionada no topo. Durante o percurso, os ponteiros proximo e anterior de cada carta são invertidos, modificando fisicamente a ordem de encadeamento dos elementos da lista duplamente ligada. Ao final, os ponteiros topo e fundo são trocados

Como a operação VIRA também deve ser registrada no histórico de operações, é necessário armazenar essa ação para permitir sua posterior reversão por meio da operação DESFAZ.

```
1
2  boolean VIRA(Baralho *baralho){
3      if(baralho == NULL){
4          return FALSE;
5      }
6      else{
7          Carta *carta_atual = baralho -> topo;
8          //O(n)
9          for(int i = 0; i < baralho->tamanho_baralho; i++){
10             Carta *carta_proxima = carta_atual -> proximo;
11             ....
12             }
```

CARTA devolve a carta que está na posição p. A lógica aqui bebe muito da fonte de VIRA. Inicializamos uma carta que começa no topo, percorremos toda a lista usando um for que vai até posição - 1 e enquanto o loop acontece, avançamos uma carta na lista, ao final retornamos a carta que está na posição p.

MOSTRA é uma das funções mais simples, começamos no topo, percorremos até o tamanho do baralho usando um for printando cada uma das cartas e avançamos para a próxima carta, ao final do loop, imprimimos um espaçamento.

POSICAO retorna a posição de uma determinada carta no baralho, utilizando seu número como critério de busca. Inicialmente, criamos um ponteiro auxiliar que aponta para o topo e um contador de posição, inicializado em 1. Em seguida, percorremos a lista utilizando um while loop, avançando para o próximo nó e incrementando o contador até encontrar a carta desejada. Sua complexidade é $O(n)$ no pior caso e $O(1)$ no melhor caso.

CORTA é uma função que reorganiza o baralho. Ela coloca uma carta específica no topo e reorganiza os elementos da lista. Percorremos do topo até a carta que queremos. Caso ela não seja encontrada, a operação é interrompida. Caso contrário, os ponteiros

são atualizados de modo que a carta encontrada se torne o novo topo, enquanto a carta que a antecedia passa a ser o novo fundo.

VIRATOPO inverte a ordem das primeiras k cartas do baralho, fazendo com que a carta que ocupava a posição k passe a ser a primeira. Percorre a lista até encontrar a carta na posição k e guarda um ponteiro para a carta seguinte. Depois, inverte os ponteiros proximo e anterior das primeiras k cartas, alterando a ordem em que estão encadeadas.No final, reconecta o trecho invertido ao restante do baralho e atualiza os ponteiros topo e fundo, quando necessário.

SOBE recebe o número de uma carta e a coloca ela no topo do baralho. No ínicio, percorre a lista utilizando um ponteiro auxiliar e um contador de posição, até encontrar a carta desejada. Caso ela não seja encontrada, a operação é interrompida. Se o registro estiver habilitado, sua posição original é armazenada no histórico, permitindo que a operação seja desfeita posteriormente. Caso a carta já esteja no topo, nenhuma alteração é necessária. Caso contrário, seus nós vizinhos são conectados entre si, retirando a carta de sua posição atual. Em seguida, os ponteiros anterior e proximo da carta são atualizados para inseri-la no início da lista, e o ponteiro topo é atualizado para apontar para ela. Se a carta removida ocupava o fundo, o ponteiro fundo também é atualizado.

A CARTA recebe uma posição e retorna o número da carta armazenada nessa posição. Logo depois, cria um ponteiro auxiliar que aponta para o topo do baralho e utiliza um for para avançar pela lista até alcançar a posição desejada. No final, retorna o identificador da carta encontrada. Caso a posição seja inválida, retorna -1.

DESFAZ é uma das operações mais curiosas implementadas até aqui. Ela permite reverter uma operação realizada no baralho usando o histórico, que faz parte da struct do baralho. Ela chama REMOVERULTIMO, que retira o registro mais recente e recupera suas informações. Caso não exista uma operação registrada, ela retorna FALSE. Se encontrar uma operação registrada, utiliza um switch para determinar qual operação deve ser desfeita. Caso a operação seja CORTA, VIRA ou VIRATOPO, a função chama essas mesmas funções novamente, revertendo, assim, a operação. Caso a operação seja SOBE, utiliza a função auxiliar reposicionar, que devolve a carta à sua posição original, informação que está guardada no histórico.

2 JUSTIFICATIVA

Na especificação, temos que o tamanho máximo de um baralho é 100000, o que, inicialmente, parece ser um ponto positivo para estruturas estáticas sequenciais, já que cria um limite. Entretanto, acreditamos que a menos que na maioria dos casos o baralho tenha perto de 100000 cartas, não vale a pena criar um vetor com tal tamanho, pois grande parte da memória nunca seria usada. Imaginando um jogo comum de baralho, em que o programa serviria como a mão de um jogador, é bem improvável que sejam

necessárias tantas cartas.

Por conta disso, escolhemos apenas estruturas dinâmicas encadeadas.

As estruturas escolhidas para fazer comparações e então decidir a ganhadora foram: Lista, Deque, Pilha. Percebemos que tudo que dá para fazer com Pilha, dá pra fazer com Deque, e tudo que dá pra fazer com Deque, dá pra fazer com Lista. Por isso, nossa tese era a de que a lista possuiria as menores complexidades, e então seria a estrutura ganhadora. Entretanto, como é possível ver na tabela abaixo, todas as operações possuem complexidade linear, o que levanta a questão de se a lista realmente é a melhor.

Tabela 1 – Ordem de complexidade de tempo

Operação	**Pilha**	**Deque**	**Lista**
CORTA	O(n)	O(n)	O(n)
SOBE	O(n)	O(n)	O(n)
POSICAO	O(n)	O(n)	O(n)
CARTA	O(n)	O(n)	O(n)
DESFAZ	O(n)	O(n)	O(n)
VIRA	O(n)	O(n)	O(n)
VIRATOPO	O(n)	O(n)	O(n)
MOSTRA	O(n)	O(n)	O(n)

Vamos fazer a análise experimental para verificar qual a estrutura ganhadora.

2.1 Medição

Obtemos a "Inclinação média", dividindo o resultado de 25k pelo de 12.5k, 50k pelo de 25k, e 100k pelo de 50k. Depois somamos e dividimos por 3, para conseguir a média. Por fim, dividimos por 2, pois o n dobra a cada medição.

Tabela 2 – Análise experimental Lista

Operação	**12500**	**25000**	**50000**	**100000**	**Inclinação média**
CORTA	0,28s	1,36s	6,51s	30,58s	2,4
SOBE	0,52s	2,86s	16,68s	1m42,06s	2,91
POSICAO	0,23s	0,82s	3,14s	12,38s	1,88
CARTA	0,31s	0,83s	3,1s	12,41s	1,73
DESFAZ	0,25s	1,11s	5,33s	23,2s	2,26

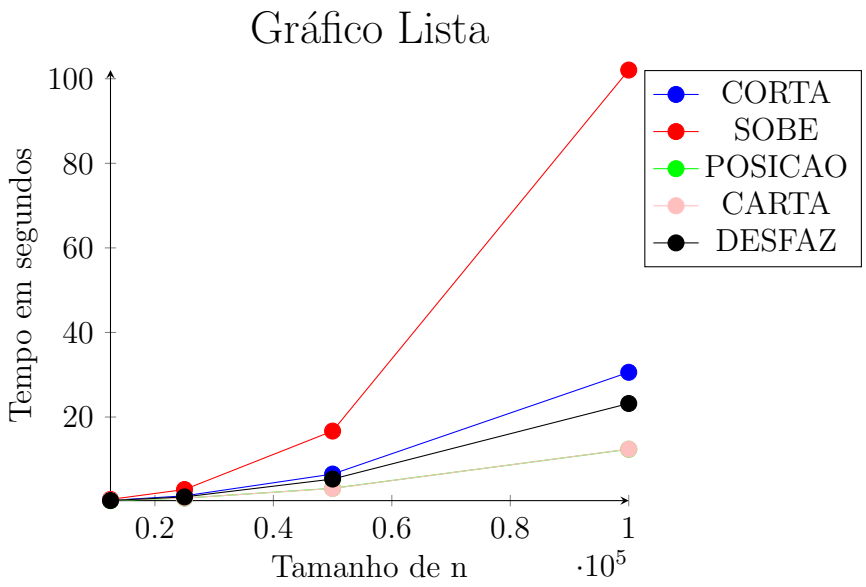


Tabela 3 – Análise experimental Pilha

Operação	**12500**	**25000**	**50000**	**100000**	**Inclinação média**
CORTA	3,17s	13s	53,71s	3m41,22s	2,06
SOBE	2,75s	12,3s	1m3,57s	5m29,88s	2,47
POSICAO	0,23s	0,88s	3,35s	13,85s	2
CARTA	0,28s	0,85s	3,25s	11,44s	1.72
DESFAZ	4,8s	19,6s	1m19,3s	5m28s	2,04

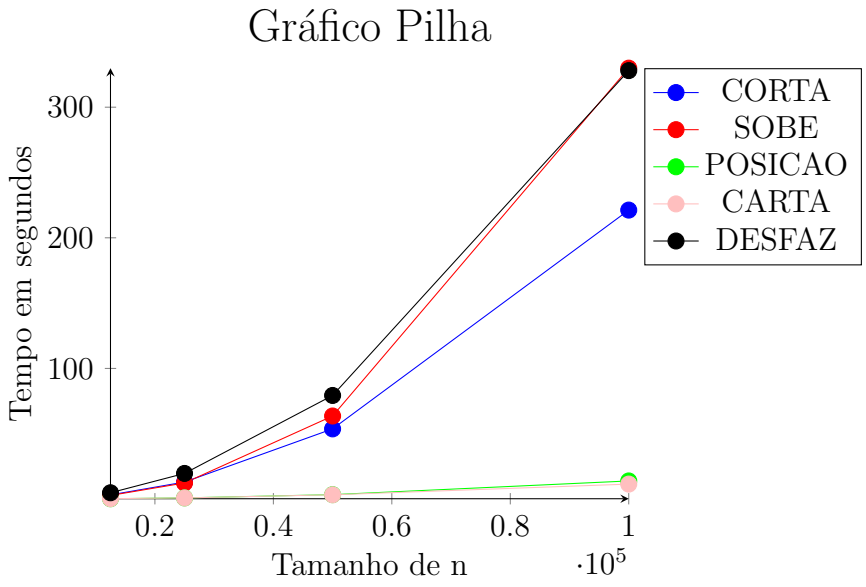
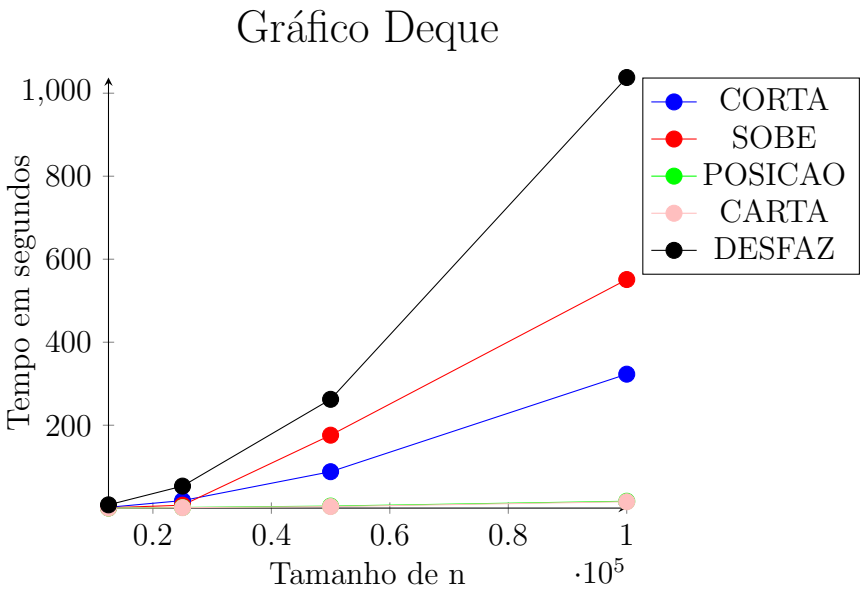


Tabela 4 – Análise experimental Deque

Operação	12500	25000	50000	100000	Inclinação média
CORTA	2,19s	18,39s	1m28s	5m23s	0,90
SOBE	1,14s	7,28s	2m56s	9m11s	1,00
POSICAO	0,43s	1,83s	5,14s	16,80s	0,90
CARTA	0,33s	1,32s	4,01s	15,61s	1,00
DESFAZ	8,14s	53,3s	4m22,4s	17m,17,8s	1,21



Como é possível ver, a lista teve o melhor desempenho. O que percebemos foi que a Pilha foi implementada de um jeito que utiliza muitas alocações. No caso de teste "TESTE DESFAZ", a Pilha faz 41 mil alocações, contra as 204 alocações da Lista. Acreditamos que é essa grande diferença que faz a Pilha se sair muito pior que a Lista na análise experimental. Além disso, vemos também que houve muitas discordâncias entre a análise assintótica e a análise experimental. Para a Lista, SOBE parece tender a O(n), enquanto que para Pilha e Deque, CORTA, SOBE e DESFAZ parecem tender a O(n). Não entendemos a razão pela qual isso acontece, porém isso ainda significa que a lista é a melhor escolha. Por isso, nossa escolha final é a Lista duplamente encadeada.

3 PONTOS FRACOS

- CARTA: Como a lista não permite acesso direto aos elementos por meio de seus índices, encontrar uma carta em uma posição específica exige percorrer de maneira sequencial, os nós a partir do topo ou do fundo. Dessa forma, a operação apresenta complexidade $O(n)$ no pior caso, sendo necessário percorrer parte significativa do baralho para localizar determinadas cartas.

- MOSTRA: A operação exige percorrer todos os elementos da lista para imprimir as cartas. Logo, sua complexidade é $O(n)$, sendo necessário visitar cada carta individualmente. Embora essa operação seja linear quando todas as cartas precisam ser exibidas, o percurso sequencial representa uma limitação em relação a estruturas que permitem acesso direto aos elementos.

- VIRA: Apesar de a lista duplamente encadeada permitir a inversão da ordem dos elementos por meio da atualização dos ponteiros `proximo` e `anterior`, a operação exige a manipulação das referências de cada nó. Um erro na atualização desses ponteiros pode comprometer a integridade da lista, provocando perda de referências ou encadeamentos incorretos. Sua complexidade é $O(n)$, uma vez que cada carta precisa ser visitada.

- CORTA: A operação exige percorrer o baralho e reorganizar os encadeamentos dos nós para modificar a posição das cartas. Apesar de a lista duplamente encadeada permitir realizar essas alterações sem a necessidade de desalocar e alocar novamente os elementos, a manipulação dos ponteiros torna a implementação mais delicada, especialmente na atualização dos limites do baralho (`topo` e `fundo`).

- GERENCIAMENTO DE MEMÓRIA: Como cada carta é armazenada em um nó alocado dinamicamente, a estrutura exige gerenciamento individual de memória. Além disso, cada nó precisa armazenar dois ponteiros, `proximo` e `anterior`, aumentando o consumo de memória em comparação com uma lista simplesmente encadeada.

4 CASOS DE TESTE

TESTE DESFAZ: Modifique o gerador para que quando a rajada for DESFAZ, n operações (da trinca) sejam feitas e depois n DESFAZ. Use agora o gerador com $n = 100$ e adicione à entrada um MOSTRA antes de FIM. Use essa entrada no programa principal.
OBS: É possível fazer tudo isso apenas usando o gerador com "JOGO 100".
Saída esperada: baralho em ordem crescente.

4.1 Testes de borda

Iremos focar em testar as operações CORTA, SOBE, VIRATOPO, POSICAO e CARTA, que necessitam de um segundo argumento (e que será a "borda"dos nossos testes).

CASO 1: BARALHO n, $n = 1$ ou $n = 100000$ ou $1 < n < 100000$ → OPERAÇÃO 1 ou OPERAÇÃO n → MOSTRA → DESFAZ → MOSTRA.

OBS: "OPERAÇÃO"é uma das 3 operações: CORTA, SOBE, VIRATOPO.
Saída esperada no primeiro MOSTRA:

- CORTA 1: baralho em ordem crescente
- CORTA n: carta n no início, depois baralho em ordem crescente
- SOBE 1: baralho em ordem crescente
- SOBE n: carta n no início, depois baralho em ordem crescente (até n-1)
- VIRATOPO 1: baralho em ordem crescente-
- VIRATOPO n: baralho em ordem decrescente

No segundo MOSTRA, todos os baralhos precisam estar em ordem crescente.

CASO 2: Use o gerador para conseguir uma entrada de rajadas (sem ser DESFAZ), com $n = 1$ ou $n = 100000$ ou $1 < n < 100000$, adicione um MOSTRA antes de FIM, verifique quais são as cartas no início e no fim, e então adicione CARTA 1, POSICAO cartaInicio, CARTA n, POSICAO cartaFim
A saída esperada será "cartaInicio, 1, cartaFim, n".

TESTE DUAS OPERAÇÕES: BARALHO n, $1 \leq n \leq 100000 \rightarrow$ OPERAÇÃO k, $1 \leq k \leq n \rightarrow$ repita a ultima operacao $\rightarrow$ DESFAZ $\rightarrow$ MOSTRA $\rightarrow$ DESFAZ $\rightarrow$ MOSTRA
OBS: "OPERAÇÃO"é CORTA ou SOBE
Saída esperada: no primeiro MOSTRA, o baralho precisa estar exatamente igual a quando é feito "OPERAÇÃO k"uma vez. No segundo MOSTRA, o baralho precisa estar em ordem crescente